Oracle RAC — Complete Basics Roadmap

We will learn in this order:

1. What is RAC?
2. RAC Architecture
3. Database vs Instance
4. RAC Nodes
5. Public Network
6. Private Interconnect
7. VIP
8. SCAN
9. OCR
10. OLR
11. Voting Disk
12. Clusterware / Grid Infrastructure
13. ASM
14. RAC Services
15. Cache Fusion
16. GCS & GES
17. `V$` vs `GV$`
18. RAC startup/shutdown
19. RAC node failure
20. Instance failure
21. Node eviction
22. Split Brain
23. Load balancing
24. Connection Failover
25. RAC + Data Guard
26. RAC troubleshooting
27. Daily RAC production checks

---

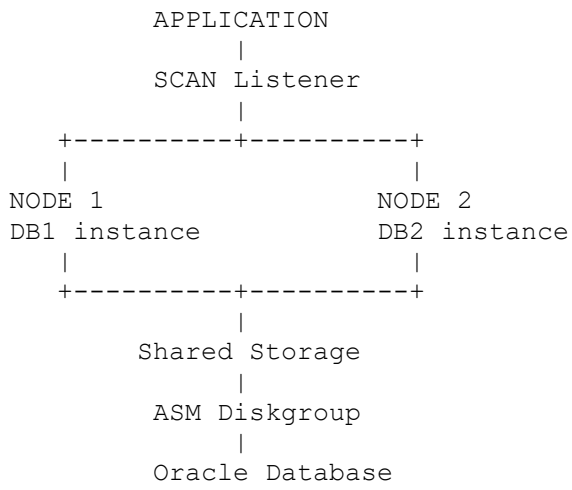
1. What is Oracle RAC?

RAC = Real Application Clusters.

In simple words:

RAC allows multiple servers/instances to access the same Oracle database simultaneously.

For example:



Here:

- Node 1 → Instance 1
- Node 2 → Instance 2
- Both instances access **the same database**
- ASM manages the shared storage.

Why do companies use RAC?

Mainly for:

High Availability + Scalability + Load Distribution

Suppose:

```
NODE 1 → Instance 1
NODE 2 → Instance 2
```

If Node 1 fails:

```
NODE 1 ✗

NODE 2 ✓
```

The database can continue serving users through Node 2, depending on the application/service configuration.

That's the basic idea of RAC.

---

2. Database vs Instance

This is **very important**.

Database

A database is the physical files:

```
Datafiles
Control files
Online redo logs
Tempfiles
```

Instance

An instance consists mainly of:

```
SGA
+
Oracle background processes
```

For a single-instance database:

```
Instance
   |
   +---- Database
```

For RAC:

```
Instance 1 ----\
                \
                 Database
                /
Instance 2 ----/
```

So **multiple instances access one database**.

Example

Suppose:

```
Database = PROD
```

You might have:

```
PROD1 → Instance on Node 1
PROD2 → Instance on Node 2
```

Both belong to the same RAC database:

```
PROD
├── PROD1
```

```
└── PROD2
```

---

3. RAC Node

A **node is a physical/virtual server participating in the RAC cluster.**

Example:

```
Node 1
Hostname: prod-db01

Node 2
Hostname: prod-db02
```

Each node normally has:

- Operating system
- Grid Infrastructure
- Oracle software
- RAC instance
- Network connectivity
- Access to shared storage

Example:

```
prod-db01 → PROD1
prod-db02 → PROD2
```

---

4. Public Network

The public network is used for normal communication with the server.

For example:

```
Application
     |
Public Network
     |
RAC Node
```

DBA/admin access can also happen through the public network.

Example:

```
ssh oracle@prod-db01
```

---

5. Private Interconnect

This is **one of the most important RAC concepts**.

The private interconnect is a dedicated network used for communication **between RAC instances/nodes**.

Example:

```
NODE 1                    NODE 2
Instance 1                Instance 2
    |                         |
    +------ Private ----------+
           Interconnect
```

It is heavily involved in RAC's **Cache Fusion** mechanism.

Why is it important?

Suppose Instance 1 has a block in its buffer cache and Instance 2 needs that block.

Instead of necessarily reading it from disk, RAC can transfer the block through the **private interconnect**.

Therefore:

Private interconnect performance is critical for RAC performance.

---

6. VIP

VIP = Virtual IP.

Each RAC node normally has a VIP.

Example:

```
Node 1
Public IP → 10.10.10.11
VIP       → 10.10.10.21

Node 2
Public IP → 10.10.10.12
VIP       → 10.10.10.22
```

Why VIP?

It helps provide **faster connection failure detection/failover**.

If Node 1 fails, its VIP can become unavailable immediately rather than waiting for normal network timeout behavior.

This helps applications fail over to another available node/service more quickly.

---

7. SCAN

SCAN = Single Client Access Name.

This is another major RAC concept.

Instead of applications needing to know every RAC node:

```
Node1
Node2
Node3
```

they can connect using the SCAN name.

Example:

```
prod-scan.company.com
```

SCAN normally resolves to multiple IP addresses.

Conceptually:

```
Application
     |
     v
PROD-SCAN
     |
 +---+---+
 |   |   |
L1  L2  L3
```

SCAN listeners help direct client connections toward the appropriate database service.

Interview answer

SCAN provides a single network name for clients to connect to an Oracle RAC cluster and helps provide connection load balancing and high availability.

---

8. OCR

OCR = Oracle Cluster Registry.

Think of OCR as the **cluster's configuration information repository**.

It stores information about cluster resources and configuration, such as:

- RAC database resources
- Services
- VIPs
- SCAN resources
- Network resources
- Cluster configuration

In simple terms:

OCR tells Clusterware what resources exist and how they are configured.

---

9. OLR

OLR = Oracle Local Registry.

OLR contains information that is specific to a particular node.

Simple difference:

```
OCR → Cluster-wide configuration

OLR → Node-local configuration
```

For interview:

OCR stores cluster-level resource configuration, while OLR stores local node-specific Grid Infrastructure information.

---

10. Voting Disk

Voting disks are used by Oracle Clusterware to maintain **node membership**.

Think of them as helping Clusterware answer:

"Which nodes are currently members of the cluster?"

They are important for preventing situations where multiple nodes incorrectly believe they own the cluster.

Example:

```
      Cluster
     /       \
Node1         Node2
     \       /
     Voting
      Disk
```

If there is a serious communication problem, Clusterware uses voting information as part of determining cluster membership and may evict a node to protect the cluster.

---

11. Grid Infrastructure / Clusterware

Oracle RAC requires **Grid Infrastructure**.

It includes important components such as:

```
Oracle Clusterware
+
ASM
```

Clusterware manages the cluster resources.

For example:

```
Database
Instance
VIP
SCAN
Listener
Service
ASM
```

You can see cluster resources using:

```
crsctl stat res -t
```

or:

```
crsctl status resource -t
```

---

12. ASM

ASM = Automatic Storage Management.

ASM is commonly used for RAC shared storage.

Example:

```
RAC Node 1 ----\
                \
                 ASM Diskgroup
                /
RAC Node 2 ----/
```

Typical disk groups:

```
+DATA
+FRA
```

`+DATA` might contain:

- Datafiles
- Control files
- Online redo logs

`+FRA` might contain:

- Archive logs
- RMAN backups
- Flashback-related files

Check ASM:

```
SELECT name, state, type, total_mb, free_mb
FROM v$asm_diskgroup;
```

---

13. RAC Services

Don't think only in terms of instances.

Production applications generally connect to **services**.

Example:

```
PROD database

PROD_APP service
PROD_REPORT service
```

A service can be configured to run on particular RAC instances.

For example:

```
PROD_APP

Preferred:
PROD1

Available:
```

```
PROD2
```

If PROD1 fails, the service can be relocated/fail over to PROD2 depending on configuration.

Check:

```
srvctl status service -d PROD
```

---

14. Cache Fusion ⭐

This is probably the **most important RAC concept for interviews**.

Each RAC instance has its own buffer cache:

```
Instance 1
Buffer Cache 1

Instance 2
Buffer Cache 2
```

But both access the same database.

Suppose:

```
Instance 1 has Block A
```

Instance 2 needs Block A.

RAC can transfer the block between instances through the **private interconnect**.

This mechanism is called:

Cache Fusion

Conceptually:

```
Instance 1
Buffer Cache
     |
     | Block transfer
     | Private Interconnect
     v
Instance 2
Buffer Cache
```

This avoids unnecessary physical disk reads in many situations.

---

15. GCS and GES

GCS

GCS = Global Cache Service

It manages access to cached data blocks across RAC instances.

GES

GES = Global Enqueue Service

It coordinates certain global locks/enqueues across instances.

Simple interview answer:

GCS manages global cache resources and block access across RAC instances, while GES manages global enqueue resources.

---

16. V$ vs GV$

This is something you will use **every day as a RAC DBA**.

V$

Shows information for the **current instance**.

Example:

```
SELECT instance_name
FROM v$instance;
```

GV$

Shows information across **all RAC instances**, with `INST_ID`.

Example:

```
SELECT inst_id,
       instance_name,
       status,
       host_name
FROM gv$instance;
```

Output could be:

```
INST_ID   INSTANCE_NAME   STATUS
```

```
1          PROD1            OPEN
2          PROD2            OPEN
```

Remember:

V$ = current instance

GV$ = global/all instances

---

17. RAC Startup

A simplified RAC startup sequence is:

```
OS
 ↓
Grid Infrastructure
 ↓
Clusterware
 ↓
ASM
 ↓
Database Instance
 ↓
Services
```

You should know the difference between starting:

- Cluster
- ASM
- Database
- Instance
- Service

For example:

```
srvctl start database -d PROD
```

Check:

```
srvctl status database -d PROD
```

---

18. RAC Shutdown

Never treat RAC shutdown as simply:

```
shutdown immediate;
```

In production, first understand whether you're shutting down:

- One instance
- The whole database
- A service
- A node
- Grid Infrastructure

For a controlled instance operation, `srvctl` is generally used in RAC environments.

Example:

```
srvctl stop instance -d PROD -i PROD1
```

But **don't execute production stop/start commands without the approved procedure/change**.

---

19. What happens if one RAC node fails?

Example:

```
Before:

NODE1 → PROD1
NODE2 → PROD2

Both healthy
```

Node 1 crashes:

```
NODE1 ✕

NODE2 → PROD2 ✓
```

Clusterware detects the failure.

Depending on service configuration:

```
PROD_APP
     ↓
moves/fails over
     ↓
PROD2
```

Applications may reconnect through the available instance/service.

The exact behavior depends on the application's connection configuration and RAC service configuration.

---

20. Instance Failure

A node and an instance are not exactly the same thing.

Example:

```
NODE1
  |
  +--- PROD1
```

If the Oracle instance crashes but the server remains available:

```
NODE1 ✓
PROD1 ✗
```

Clusterware can detect the failed resource and attempt recovery/restart according to configuration.

---

21. Node Eviction

Node eviction means:

Clusterware forcibly removes a problematic node from the cluster.

Why?

To protect the overall cluster and prevent data corruption/inconsistent cluster membership.

For example, severe interconnect problems can cause a node to lose communication with the rest of the cluster.

Clusterware may decide:

```
NODE1 → evict
NODE2 → remain
```

This can be scary in production, but it is a **protective mechanism**.

---

22. Split Brain

Very important interview topic.

Imagine:

```
NODE1   X   NODE2
```

The nodes cannot communicate properly.

Both might incorrectly think:

"I am the only healthy node."

That's called a **split-brain situation**.

Oracle Clusterware uses cluster membership mechanisms including voting disks and interconnect communication to prevent multiple isolated groups from independently operating as the valid cluster.

---

23. Load Balancing

RAC can distribute application connections across instances.

Example:

```
1000 connections

          SCAN
         /    \
      PROD1   PROD2
       500     500
```

The actual distribution depends on service configuration, listeners, client configuration and workload.

---

24. Connection Failover

Suppose:

```
Application
     |
   PROD1
```

PROD1 fails.

If the service/client configuration supports failover:

```
Application
     |
     X PROD1
     |
```

```
    ↓
  PROD2
```

The application can reconnect to the available instance.

This is one reason RAC is useful for high availability.

---

25. RAC + Data Guard

In many enterprise production environments you may see:

```
PRIMARY RAC
-----------------
Node1 → PROD1
Node2 → PROD2
        |
        | Data Guard
        ↓
STANDBY RAC
-----------------
Node3 → STBY1
Node4 → STBY2
```

So don't confuse:

RAC → high availability within the cluster.

Data Guard → disaster recovery/standby database technology.

They solve different problems and can be used together.

---

26. RAC Troubleshooting — Your L1 DBA Approach

Suppose an application says:

"Database is slow."

Don't immediately restart anything.

Follow:

```
1. Check database
        ↓
2. Check all RAC instances
        ↓
```

```
3. Check services
       ↓
4. Check sessions
       ↓
5. Check blocking
       ↓
6. Check wait events
       ↓
7. Check SQL_ID
       ↓
8. Check CPU/memory/I/O
       ↓
9. Check ASM
       ↓
10. Check alert/cluster logs
       ↓
11. Identify root cause
       ↓
12. Take approved action
       ↓
13. Monitor
```

Useful starting query:

```
SELECT inst_id,
       instance_name,
       status,
       host_name
FROM gv$instance;
```

Then:

```
SELECT inst_id,
       username,
       status,
       event,
       sql_id
FROM gv$session
WHERE status = 'ACTIVE';
```

---

27. Your Daily RAC Production Checklist

When you start your shift, think:

Database

```
SELECT inst_id, instance_name, status, host_name
FROM gv$instance;
```

Database role/open mode

```
SELECT inst_id, open_mode, database_role
FROM gv$database;
```

ASM

```
SELECT name, state, type, total_mb, free_mb
FROM v$asm_diskgroup;
```

Cluster

```
crsctl stat res -t
```

Database

```
srvctl status database -d PROD
```

Services

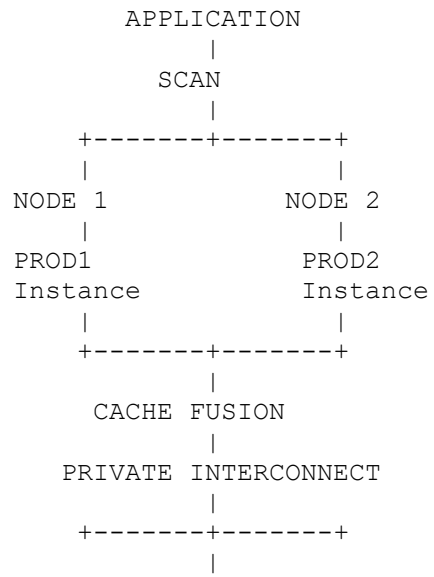
```
srvctl status service -d PROD
```

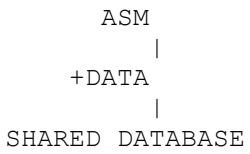
Then check:

- Alert logs
- Backup status
- Data Guard status, if applicable
- ASM free space
- Blocking sessions
- Long-running jobs
- CPU/memory
- Any overnight alerts/incidents

⭐ The RAC Mental Picture

If you remember only one diagram, remember this:





And remember these 10 words:

RAC = Multiple Instances + One Database

Node → Server

Instance → SGA + background processes

ASM → Storage management

SCAN → Client access name

VIP → Virtual IP for node connectivity/failover

Interconnect → RAC instance-to-instance communication

Cache Fusion → Global block coordination/transfer

OCR → Cluster configuration

Voting Disk → Cluster membership

What I recommend for you

Don't try to memorize all 27 topics at once.

We can do this **like a real RAC training session**:

Lesson 1 → RAC Architecture
Lesson 2 → Node & Instance
Lesson 3 → SCAN/VIP/Listeners
Lesson 4 → Private Interconnect
Lesson 5 → OCR/OLR/Voting Disk
Lesson 6 → ASM
Lesson 7 → Cache Fusion/GCS/GES
Lesson 8 → Services
Lesson 9 → Startup/Shutdown
Lesson 10 → Real production troubleshooting

🔥 MOST IMPORTANT RAC COMMANDS

These are the commands I would memorize first for an **L1 RAC DBA**.

1. Check all cluster resources

```
crsctl stat res -t
```

or:

```
crsctl status resource -t
```

This is one of your **first commands when checking RAC health**.

---

2. Check cluster

```
crsctl check cluster
```

---

3. Check cluster nodes

```
olsnodes -n
```

---

4. Check RAC database

```
srvctl status database -d PROD
```

Example:

```
Instance PROD1 is running on node prod-db01
Instance PROD2 is running on node prod-db02
```

---

5. Check database configuration

```
srvctl config database -d PROD
```

---

6. Check RAC instances from SQL ⭐⭐⭐

```
SELECT inst_id,
       instance_name,
       status,
       host_name
FROM gv$instance
ORDER BY inst_id;
```

This is one of the most useful RAC queries.

---

7. Check database open mode

```
SELECT inst_id,
       name,
       open_mode,
       database_role
FROM gv$database;
```

🔥 V$ vs GV$ — MUST REMEMBER

V$

Current instance.

```
SELECT instance_name
FROM v$instance;
```

GV$

All RAC instances.

```
SELECT inst_id,
       instance_name,
       status,
       host_name
FROM gv$instance;
```

In RAC troubleshooting, **GV$ is extremely important**.

🔥 SESSION MONITORING COMMANDS

8. Check active sessions

```
SELECT inst_id,
       sid,
       serial#,
       username,
       status,
       event,
       sql_id
FROM gv$session
WHERE status = 'ACTIVE'
ORDER BY inst_id;
```

---

9. Check blocking sessions

```
SELECT inst_id,
       sid,
       serial#,
       username,
       blocking_instance,
       blocking_session,
       event,
       sql_id
FROM gv$session
WHERE blocking_session IS NOT NULL;
```

Very important:

Don't immediately kill the blocking session.

First determine:

- Who is the user?
- Which application?
- What SQL_ID?
- How long has it been running?
- What is it blocking?
- Is it business critical?
- What does your SOP say?

🔥 SQL PERFORMANCE

10. Find SQL_ID from active sessions

```
SELECT inst_id,
       sid,
       serial#,
       username,
```

```
       sql_id,
       event,
       status
FROM gv$session
WHERE status = 'ACTIVE';
```

Then investigate that SQL_ID using your approved performance queries/tools.

🔥 ASM COMMANDS

11. Check ASM disk groups

Run from ASM instance:

```
SELECT name,
       state,
       type,
       total_mb,
       free_mb
FROM v$asm_diskgroup;
```

Look at:

```
STATE
TYPE
TOTAL_MB
FREE_MB
```

---

12. Check ASM disks

```
SELECT group_number,
       disk_number,
       name,
       path,
       mount_status,
       header_status,
       mode_status,
       state
FROM v$asm_disk
ORDER BY group_number, disk_number;
```

Important states include:

```
NORMAL
ONLINE
OFFLINE
```

The exact interpretation depends on the column and ASM situation.

🔥 SERVICE COMMANDS

13. Check services

```
srvctl status service -d PROD
```

---

14. Check service configuration

```
srvctl config service -d PROD
```

🔥 LISTENER COMMANDS

15. Check listener

```
srvctl status listener
```

---

16. Check SCAN listener

```
srvctl status scan_listener
```

🔥 RAC DATABASE CONFIGURATION

17. Check database configuration

```
srvctl config database -d PROD
```

This helps you understand how the RAC database is registered/configured.

🔥 LONG RUNNING OPERATIONS

18. Check long-running operations

```
SELECT inst_id,
       sid,
       serial#,
       opname,
       sofar,
       totalwork,
```

```
       units,
       elapsed_seconds
FROM gv$session_longops
WHERE totalwork > 0
AND sofar < totalwork
ORDER BY elapsed_seconds DESC;
```

Useful during:

- RMAN
- Large operations
- Index operations
- Data movement
- Other long-running database operations

🔥 WAIT EVENTS

19. Check non-idle waits

```
SELECT inst_id,
       event,
       COUNT(*) sessions_waiting
FROM gv$session
WHERE wait_class <> 'Idle'
GROUP BY inst_id, event
ORDER BY sessions_waiting DESC;
```

This can help you identify where sessions are waiting.

Don't assume that the event with the highest count is automatically the root cause—correlate it with workload and timing.

🔥 RAC REDO / THREADS

RAC instances normally have separate redo threads.

Check:

```
SELECT thread#,
       MAX(sequence#) max_sequence
FROM gv$archived_log
GROUP BY thread#
ORDER BY thread#;
```

This is useful when checking RAC redo/archive information.

🔥 ALERT LOG

If your environment allows querying ADR:

```
SELECT originating_timestamp,
       message_text
FROM v$diag_alert_ext
WHERE originating_timestamp >
      SYSTIMESTAMP - INTERVAL '1' HOUR
ORDER BY originating_timestamp DESC;
```

Look for:

- ORA errors
- Instance crash
- Resource problems
- Storage issues
- Connectivity problems
- Data Guard errors
- Unexpected events

□ YOUR REAL-TIME RAC TROUBLESHOOTING FLOW

Suppose application says:

"Production database is slow."

Don't restart anything.

Start with:

Step 1 — Are all instances up?

```
SELECT inst_id,
       instance_name,
       status,
       host_name
FROM gv$instance;
```

Step 2 — Is the database open?

```
SELECT inst_id,
       open_mode,
       database_role
```

```
FROM gv$database;
```

Step 3 — Are cluster resources healthy?

```
crsctl stat res -t
```

Step 4 — Are services running?

```
srvctl status service -d PROD
```

Step 5 — Check active sessions

```
SELECT inst_id,
       sid,
       serial#,
       username,
       event,
       sql_id
FROM gv$session
WHERE status = 'ACTIVE';
```

Step 6 — Check blocking

```
SELECT inst_id,
       sid,
       serial#,
       username,
       blocking_instance,
       blocking_session,
       sql_id
FROM gv$session
WHERE blocking_session IS NOT NULL;
```

Step 7 — Check waits

```
SELECT inst_id,
       event,
       COUNT(*)
FROM gv$session
WHERE wait_class <> 'Idle'
GROUP BY inst_id, event
ORDER BY COUNT(*) DESC;
```

Step 8 — Check ASM

```
SELECT name,
       state,
       total_mb,
       free_mb
FROM v$asm_diskgroup;
```

Step 9 — Check alert log

```
SELECT originating_timestamp,
       message_text
```

```
FROM v$diag_alert_ext
WHERE originating_timestamp >
      SYSTIMESTAMP - INTERVAL '1' HOUR
ORDER BY originating_timestamp DESC;
```

Step 10 — Then decide what to do

```
CHECK
  ↓
IDENTIFY
  ↓
ANALYZE
  ↓
CONFIRM
  ↓
APPROVAL / SOP
  ↓
ACTION
  ↓
VERIFY
  ↓
MONITOR
  ↓
DOCUMENT
```

⭐ 15 Commands I Want You to Memorize First

If you are starting RAC production work, memorize these first:

```
crsctl stat res -t
crsctl check cluster
olsnodes -n

srvctl status database -d PROD
srvctl config database -d PROD

srvctl status service -d PROD
srvctl config service -d PROD

srvctl status listener
srvctl status scan_listener
```

And these SQL queries:

```
SELECT inst_id, instance_name, status, host_name
FROM gv$instance;
SELECT inst_id, name, open_mode, database_role
FROM gv$database;
SELECT inst_id, sid, serial#, username, status, event, sql_id
FROM gv$session
WHERE status='ACTIVE';
SELECT inst_id, sid, serial#, username,
       blocking_instance, blocking_session, sql_id
```

```
FROM gv$session
WHERE blocking_session IS NOT NULL;
SELECT name, state, type, total_mb, free_mb
FROM v$asm_diskgroup;
```

Yes — **your 25 points are not all "RAC architecture."** They are a combination of **RAC architecture + RAC concepts + administration + troubleshooting + daily operations**.

In short, think of it like this:

- **RAC (Real Application Clusters)** is an Oracle technology where **multiple instances running on multiple nodes access one shared database** to provide **high availability and scalability**.
- Its **architecture** mainly consists of:

Nodes + RAC Instances + Shared Storage/ASM + Public Network + Private Interconnect + VIP + SCAN + Listeners + Services + Clusterware + OCR + Voting Disk

- Then, after understanding the architecture, you learn the **RAC working concepts**:

Cache Fusion + GCS/GES + Load Balancing + Connection Failover

- Then you learn **RAC administration and troubleshooting**:

Startup/Shutdown + Node Failure + Instance Failure + Node Eviction + Split Brain + V$/GV$ + Daily Monitoring + Troubleshooting

- And finally:

RAC + Data Guard = RAC for HA + Data Guard for DR.

So your notes should be divided into 4 sections:

Section	Topics
RAC Architecture	Nodes, Instance, Database, Public Network, Private Interconnect, VIP, SCAN, Listener, Services, ASM, Clusterware, OCR, OLR, Voting Disk
RAC Concepts	Cache Fusion, GCS, GES, Load Balancing, Connection Failover
RAC Administration	Startup/Shutdown, Service management, V$/GV$, node/instance management
RAC Troubleshooting & Monitoring	Node failure, Instance failure, Node eviction, Split Brain, ASM issues, blocking, performance, daily health checks

⭐ Best way to remember

Architecture = “What components make RAC?”

Concepts = “How do those components work together?”

Administration = “How do I manage RAC?”

Troubleshooting = “What do I do when something goes wrong?”